

Sistema de control de versiones (VCS)

¿Qué es?

Un sistema de control de versiones registra los cambios realizados en uno o varios archivos a lo largo del tiempo. Gracias a esto, es posible consultar el historial de un proyecto, recuperar versiones anteriores y trabajar de manera más ordenada.

¿Para qué sirve?

- Volver a versiones anteriores de uno o varios archivos.
- Recuperar una versión previa del proyecto completo.
- Comparar cambios realizados en distintos momentos.
- Identificar quién modificó por última vez una parte del código.
- Detectar quién y cuándo introdujo un error.
- Restaurar archivos dañados o eliminados.

Tipos de VCS

Los sistemas de control de versiones pueden clasificarse en tres grandes tipos:

- **Local**
- **Centralizado**
- **Distribuido**

VCS local

En un sistema de control de versiones local, el historial de cambios se almacena en la misma computadora del usuario. Es una solución simple, pero presenta limitaciones para el trabajo colaborativo y mayores riesgos ante fallos del equipo o pérdida de información.

VCS centralizado

En un sistema centralizado, existe un servidor único que almacena el historial del proyecto y desde el cual los usuarios obtienen y envían cambios. Facilita la administración del trabajo en equipo, aunque depende fuertemente de la disponibilidad del servidor central.

VCS distribuido

En un sistema distribuido, cada usuario posee una copia completa del repositorio, incluido su historial. Esto mejora la redundancia, permite realizar muchas operaciones de forma local y favorece el trabajo colaborativo sin depender permanentemente de un servidor central.

Git

¿Qué es?

Git es un sistema de control de versiones distribuido. Fue diseñado para trabajar de manera rápida, segura y eficiente, incluso en proyectos grandes y con múltiples colaboradores.

Objetivos de diseño

- Alto rendimiento.
- Diseño simple.
- Soporte para desarrollo no lineal.
- Funcionamiento completamente distribuido.
- Capacidad para manejar proyectos de gran tamaño.

Fundamentos de Git

Las ideas principales sobre las que se apoya Git son las siguientes:

- Trabaja con **copias instantáneas**, no solo con diferencias.
- La mayoría de las operaciones son **locales**.
- Garantiza la **integridad** de los datos.
- Los archivos atraviesan distintos **estados** durante el trabajo.

Almacenamiento de datos

La primera imagen representa un modelo de almacenamiento basado en **copias diferenciadas**: cada nueva versión guarda únicamente los cambios respecto de la versión anterior.

La segunda imagen muestra el enfoque basado en **copias instantáneas**, que es el utilizado por Git: en cada versión se registra una instantánea del estado de los archivos del proyecto. Esta decisión permite manejar el historial de manera más eficiente y confiable.

Fundamentos de Git (continuación)

Copias instantáneas, no diferencias

Git guarda el estado de los archivos en distintos momentos del tiempo. En lugar de almacenar únicamente las diferencias entre versiones, registra instantáneas del proyecto. Esto permite trabajar con el historial de forma más eficiente y confiable.

Operaciones locales

En Git, muchas operaciones se realizan sobre la copia local del repositorio. Esto hace que acciones como consultar el historial, comparar cambios o crear commits sean rápidas y no dependan de una conexión permanente a Internet.

Integridad: suma de verificación (SHA-1)

Git identifica cada objeto mediante una suma de verificación calculada con el algoritmo SHA-1.

- Se representa como una cadena de 40 caracteres hexadecimales (0-9 y a-f).
- Ejemplo: 24b9da6552252987aa493b52f8696cd6d3b00373

Estados

En Git, un archivo puede encontrarse en alguno de estos estados:

- **Confirmado** (*committed*): los datos ya están almacenados de manera segura en la base de datos local del repositorio.
- **Modificado** (*modified*): el archivo fue editado, pero sus cambios todavía no se incorporaron al próximo commit.
- **Preparado** (*staged*): el archivo modificado fue marcado para incluirse en la próxima confirmación.

Estados: flujo de trabajo

La imagen representa los tres espacios principales del flujo de trabajo de Git. El **working directory** es el directorio de trabajo donde se crean y modifican los

archivos. La **staging area** es el área de preparación donde se seleccionan los cambios que formarán parte del próximo commit. El **.git directory (repository)** es la base de datos local del repositorio, donde Git almacena de manera permanente la información confirmada.

Comandos básicos

Configuración inicial

Definir el nombre del autor:

```
git config --global user.name "Nombre y apellido"
```

Definir el correo electrónico del autor:

```
git config --global user.email "correo@email.com"
```

Definir el editor de texto predeterminado:

```
git config --global core.editor notepad
```

Mostrar la configuración actual:

```
git config --list
```

Crear un repositorio

Inicializar un nuevo repositorio Git:

```
git init
```

Consultar el estado

Mostrar el estado completo del repositorio:

```
git status
```

Mostrar una versión resumida del estado:

```
git status -s
```

Estados de archivos

La imagen muestra los posibles estados por los que puede pasar un archivo dentro del repositorio. Un archivo **untracked** es un archivo nuevo que Git todavía no está siguiendo. Un archivo **unmodified** no tiene cambios respecto de la última versión confirmada. Un archivo **modified** fue editado, pero esos cambios aún no fueron preparados. Un archivo **staged** ya fue agregado al área de preparación y está listo para formar parte del próximo commit.

Otros comandos frecuentes

Agregar archivos al área de preparación

`git add archivo`

Confirmar cambios

`git commit -m "mensaje"`

Agregar y confirmar en un solo paso

`git commit -a -m "mensaje"`

Renombrar archivos

`git mv nombre_viejo nombre_nuevo`

Modificar el último commit

Se utiliza, por ejemplo, para corregir el mensaje del último commit o agregar archivos que faltaron incluir:

`git commit --amend`

Quitar un archivo del área de preparación

`git restore --staged archivo`

Descartar cambios en un archivo modificado

`git restore archivo`

Consultar el historial

Mostrar el historial completo:

`git log`

Mostrar solo el último commit:

`git log -1`

Mostrar los últimos dos commits con sus diferencias:

`git log -p -2`

Mostrar el historial de un archivo específico:

`git log archivo`

Clonar un repositorio

git clone repositorio

Trabajar con repositorios remotos

Ver los repositorios remotos configurados:

git remote -v

Agregar un repositorio remoto:

git remote add nombre repositorio

Descargar cambios desde un repositorio remoto:

git fetch nombre

Descargar y fusionar cambios del repositorio remoto:

git pull origin master

Enviar cambios al repositorio remoto:

git push origin master

¿Qué es la World Wide Web?

La **World Wide Web (WWW)** es un sistema que permite publicar, acceder y relacionar información a través de Internet mediante **páginas y aplicaciones web**.

Es importante no confundir ambos conceptos:

- **Internet** es la red que conecta computadoras y otros dispositivos entre sí
- **La Web** es uno de los servicios que funciona sobre Internet
- Para usar la Web normalmente se emplea un **navegador**
- Los recursos web se identifican mediante una **URL**

En términos simples, la Web permite que una persona, desde un celular, una notebook o una PC, solicite información a un servidor y la visualice en forma de texto, imágenes, audio, video o interfaces interactivas.

¿Cómo funciona la web?

Cuando una persona abre un sitio web, ocurre una secuencia de pasos entre el cliente, la red y el servidor.

1. Se escribe una dirección web o se hace clic en un enlace

2. El navegador interpreta esa dirección como una **URL**
 - La URL indica a qué recurso se quiere acceder
 - Por ejemplo: `https://www.ejemplo.com/productos`
3. La computadora necesita averiguar a qué máquina conectarse
 - Cada equipo conectado a Internet tiene una **dirección IP**
 - Como las personas recuerdan mejor nombres que números, se usan nombres de dominio como `www.ejemplo.com`
 - Los **DNS (Domain Name System)** traducen ese nombre a una IP
4. Con la IP obtenida, el navegador se comunica con el servidor y envía una **solicitud HTTP**
5. El servidor recibe la solicitud, la procesa y responde
 - Puede indicar que el recurso no existe
 - Puede redirigir al cliente a otra dirección
 - Puede devolver archivos ya existentes, como HTML, CSS, imágenes o videos
 - Puede ejecutar un programa para generar una respuesta dinámica
6. El navegador interpreta la respuesta y construye la página que ve la persona usuaria
 - Normalmente combina HTML, CSS, JavaScript y otros recursos
 - Distintos navegadores pueden mostrar pequeñas diferencias

En resumen, la Web funciona mediante un modelo de **pedido y respuesta**: el cliente solicita un recurso y el servidor responde.

Lado del cliente

El **lado del cliente** reúne todo lo que ocurre en el dispositivo de la persona usuaria una vez que recibe la respuesta del servidor.

- Se ejecuta principalmente en el navegador
- Incluye la estructura, la presentación y parte del comportamiento de la página
- Intervienen tecnologías como HTML, CSS y JavaScript

- También se lo conoce como **front-end**

El objetivo del lado del cliente es mostrar información e interactuar con la persona usuaria de forma clara, rápida y cómoda.

Lado del servidor

El **lado del servidor** agrupa los procesos que ocurren en una computadora remota encargada de recibir solicitudes y devolver respuestas.

- Atiende solicitudes enviadas por clientes
- Puede validar datos, consultar bases de datos o ejecutar lógica de negocio
- Puede exponer servicios como APIs, autenticación o almacenamiento
- También se lo conoce como **back-end**

En muchos sistemas, el cliente y el servidor trabajan en conjunto: el cliente presenta la interfaz y el servidor administra la información y las reglas del sistema.

Tecnologías del lado del cliente

Navegadores

El navegador es el programa que solicita recursos web, interpreta su contenido y lo muestra en pantalla. Además, administra pestañas, historial, favoritos, seguridad, descargas y herramientas de desarrollo.

Algunos navegadores populares son:

- Chrome
- Edge
- Firefox
- Brave
- Safari
- Opera

Aunque todos cumplen una función similar, no siempre interpretan exactamente igual todos los estándares. Por eso, al desarrollar sitios web, es importante probar en más de un navegador.

HTML

HTML significa **HyperText Markup Language**. Es el lenguaje de marcado que se utiliza para definir la **estructura** de una página web.

Con HTML se organizan los distintos elementos del contenido, por ejemplo:

- títulos y subtítulos
- párrafos y enlaces
- imágenes, listas y tablas
- formularios, botones y secciones

HTML no se encarga del diseño visual detallado ni de la lógica compleja. Su rol principal es describir **qué es cada parte del contenido** y cómo se relaciona con las demás.

La versión actual del estándar es conocida como **HTML5**, que incorporó etiquetas semánticas y soporte mejorado para multimedia y formularios.

Ejemplo de HTML

```
<!DOCTYPE html>

<html>

<head>

<title>Título de la página</title>

</head>

<body>

El contenido de la página va aquí

</body>

</html>
```

CSS

CSS significa **Cascading Style Sheets**. Es el lenguaje que se utiliza para definir la **presentación visual** de un documento HTML.

Con CSS se controlan aspectos como:

- colores
- tipografías
- tamaños y espaciados
- bordes, fondos y posiciones
- distribución de elementos en la pantalla

Gracias a CSS es posible separar el **contenido** de su **presentación**. Esto facilita el mantenimiento, mejora la reutilización y permite adaptar un mismo contenido a diferentes dispositivos.

La denominación "en cascada" hace referencia a que varias reglas pueden afectar a un mismo elemento, y el navegador debe determinar cuáles tienen prioridad.

Ejemplo de CSS

```
body{  
  
background-color: #f0f0f0;  
  
font-family: Arial, sans-serif;  
  
}  
  
h1 {  
  
color: #0066cc;  
  
text-align: center;  
  
}  
  
p {  
  
color: #333;  
  
font-size: 18px;  
  
text-align: center;  
  
}
```

JavaScript

JavaScript es un lenguaje de programación que permite agregar **comportamiento** e **interactividad** a una página web.

Se utiliza, por ejemplo, para:

- responder a eventos como clics o pulsaciones de teclas
- validar datos ingresados en formularios
- modificar contenido y estilos sin recargar toda la página
- comunicarse con servidores para obtener o enviar datos

En el contexto del desarrollo web inicial, JavaScript se ejecuta principalmente en el navegador. Junto con HTML y CSS forma la base del desarrollo del lado del cliente.

Aunque también existe JavaScript del lado del servidor, en este tema nos interesa sobre todo su uso en el navegador para construir interfaces dinámicas.

Ejemplo de JavaScript

```
console.log("Hola mundo")
```

¿Qué es HTML?

HTML significa **HyperText Markup Language**. Es el lenguaje de marcado que se utiliza para definir la **estructura** de una página web.

Cuando una persona abre un sitio en el navegador, gran parte de lo que ve comienza como un documento HTML. Ese documento le indica al navegador qué partes del contenido son títulos, párrafos, enlaces, imágenes, listas, tablas, formularios o secciones.

HTML **no es un lenguaje de programación**. No se usa para expresar algoritmos, decisiones o repeticiones como ocurre en lenguajes como JavaScript, Python o Java. Su función principal es **describir y organizar información**.

En desarrollo web suele trabajarse con tres tecnologías principales:

- **HTML**: estructura y significado del contenido
- **CSS**: presentación visual
- **JavaScript**: comportamiento e interactividad

Una forma simple de pensarlo es esta:

- HTML dice **qué es** cada cosa

- CSS dice **cómo se ve**
- JavaScript dice **cómo se comporta**

¿Por qué se dice que es un lenguaje de marcado?

Se lo llama lenguaje de marcado porque usa **marcas** o **etiquetas** para delimitar partes del contenido y darles un significado.

Por ejemplo, si escribimos:

```
<p>Esto es un párrafo.</p>
```

el texto sigue siendo el mismo, pero ahora el navegador sabe que debe tratarlo como un párrafo.

Del mismo modo:

```
<h1>Programación III</h1>
```

indica que ese texto es un encabezado principal.

Estructura general de un documento HTML

<!DOCTYPE html>

La declaración <!DOCTYPE html> indica al navegador que el documento está escrito siguiendo el estándar actual de HTML.

- Debe aparecer al principio del archivo
- No es una etiqueta HTML común
- Ayuda al navegador a interpretar correctamente el documento

<html>

La etiqueta <html> envuelve a todo el documento HTML.

Suele incluir el atributo lang para indicar el idioma principal del contenido:

```
<html lang="es"></html>
```

Esto mejora la accesibilidad, el trabajo de los lectores de pantalla y ciertas funciones del navegador.

<head>

La sección <head> contiene **metadatos**, es decir, información sobre el documento que no forma parte del contenido visible principal.

Por ejemplo:

- el título de la página
- la codificación de caracteres
- metadatos para buscadores
- enlaces a hojas de estilo
- referencias a scripts

Un ejemplo típico:

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
<title>Mi primera página</title>
```

```
</head>
```

```
<body>
```

La sección `<body>` contiene el contenido que la persona usuaria ve en la pantalla:

- títulos
- párrafos
- imágenes
- enlaces
- listas
- tablas
- formularios

Elementos, etiquetas y atributos

Estos tres conceptos se relacionan, pero no son exactamente lo mismo.

Etiqueta

Una **etiqueta** es la marca escrita entre corchetes angulares.

Por ejemplo:

<p></p>

Elemento

Un **elemento HTML** es la unidad completa: etiqueta de apertura, contenido y etiqueta de cierre.

<p>Esto es un párrafo.</p>

En este caso, el elemento es todo el bloque.

Atributo

Un **atributo** agrega información adicional al elemento.

Ir a Google

Aquí:

- a es el nombre del elemento
- href es un atributo
- "https://www.google.com" es el valor de ese atributo

Los atributos suelen escribirse:

- dentro de la etiqueta de apertura
- con el formato nombre="valor"
- entre comillas

Elementos contenedores y elementos vacíos

Algunos elementos contienen otros elementos o texto. Por ejemplo:

<p>Este párrafo contiene texto.</p>

Otros elementos no envuelven contenido y se usan de manera independiente. Se los suele llamar **elementos vacíos**.

Ejemplos frecuentes:

-

-
- <meta />
- <input />

Por ejemplo:

Anidación

Los elementos HTML pueden colocarse unos dentro de otros. A eso se lo llama **anidación**.

<body>

<p>Este párrafo está dentro del body.</p>

</body>

También pueden anidarse varios niveles:

<section>

<article>

<h2>Noticia</h2>

<p>Contenido de la noticia.</p>

</article>

</section>

Regla importante

El elemento que se abre primero debe cerrarse último.

Correcto:

<p>Texto con énfasis importante.</p>

Incorrecto:

<p>Texto con énfasis importante</p>

Cuando las etiquetas se cierran en un orden incorrecto, el documento queda mal formado y el navegador puede interpretarlo de manera inesperada.

Espacios, saltos de línea e indentación

En HTML, varios espacios seguidos y muchos saltos de línea suelen colapsarse al mostrarse en pantalla. Esto significa que el navegador normalmente no respeta el formato visual del código fuente como si fuera texto literal.

Por ejemplo:

`<p>Una oración Otra oración Tercera oración</p>`

se visualizará como una sola línea de texto continuo.

La indentación y los saltos en el archivo sirven sobre todo para que el código sea más legible para quien lo escribe.

Elementos de uso general

Encabezados

Los encabezados se definen con las etiquetas `<h1>` a `<h6>`.

- `<h1>` representa el encabezado más importante
- `<h6>` representa el de menor jerarquía

`<h1>Título principal</h1>`

`<h2>Subtítulo</h2>`

`<h3>Sección interna</h3>`

No conviene usarlos solo porque "se ven más grandes" o "más chicos". Deben reflejar la **jerarquía lógica** del contenido.

Párrafos

Los párrafos se definen con la etiqueta `<p>`.

`<p>Esto es un párrafo.</p>`

`<p>Esto es otro párrafo.</p>`

Cada `<p>` representa una unidad de texto separada.

Saltos de línea

La etiqueta `<br />` inserta un salto de línea sin crear un nuevo párrafo.

`<p>`

Línea 1`<br />`

Línea 2`<br />`

Línea 3

`</p>`

No debe usarse para separar bloques enteros de contenido ni para "acomodar" el diseño. Para eso se utilizan otras etiquetas y, más adelante, CSS.

Texto preformateado

La etiqueta `<pre>` permite mostrar texto respetando espacios y saltos de línea.

`<pre>`

Línea 1

Línea 2 con indentación

Línea 3 con más indentación

`</pre>`

Es útil cuando se quiere mostrar:

- código
- salidas de consola
- texto donde el espaciado tiene significado

Formato básico de texto

HTML ofrece etiquetas para marcar distintos tipos de énfasis o significado en el texto.

`<p>`

Texto `<strong>importante</strong>`, texto `<em>enfaticado</em>` y texto

`<mark>resaltado</mark>`.

`</p>`

Algunas etiquetas frecuentes son:

- `<strong>`: importancia fuerte
- `<em>`: énfasis
- `<mark>`: texto resaltado
- `<small>`: texto secundario o de menor importancia
- `<sup>`: superíndice
- `<sub>`: subíndice
- `<del>`: contenido eliminado
- `<ins>`: contenido insertado

También existen `<b>` e `<i>`, pero en general conviene priorizar etiquetas con más significado semántico, como `<strong>` y `<em>`.

Enlaces

Los enlaces se definen con la etiqueta `<a>`.

```
<a href="https://www.google.com">Ir a Google</a>
```

El atributo `href` indica el destino.

Enlace absoluto

Apunta a una dirección completa:

```
<a href="https://www.uncoma.edu.ar">Sitio de la universidad</a>
```

Enlace relativo

Apunta a otro recurso dentro del mismo sitio o proyecto:

```
<a href="contacto.html">Ir a contacto</a>
```

Abrir en otra pestaña

```
<a href="https://www.github.com" target="_blank">GitHub</a>
```

`target="_blank"` suele abrir el recurso en otra pestaña o ventana.

Buenas prácticas para enlaces

- El texto del enlace debe describir a dónde lleva
- Conviene evitar frases ambiguas como "hacé clic acá"
- Si el enlace abre un recurso externo, puede ser útil aclararlo en el texto

Imágenes

Las imágenes se insertan con la etiqueta `<img />`.

```

```

Algunos atributos frecuentes son:

- `src`: ubicación de la imagen
- `alt`: texto alternativo
- `width`: ancho
- `height`: alto

¿Para qué sirve alt?

El atributo alt es importante por varios motivos:

- mejora la accesibilidad
- describe la imagen si no puede cargarse
- ayuda a tecnologías de asistencia

El texto alternativo debe describir la imagen cuando esa descripción aporta información relevante.

Rutas absolutas y relativas

Cuando un atributo apunta a un recurso, como ocurre con href o src, la ruta puede ser absoluta o relativa.

Ruta absoluta

Incluye la dirección completa:

```

```

Ruta relativa

Se interpreta a partir de la ubicación del archivo actual:

```

```

En proyectos reales, las rutas relativas son muy comunes para enlazar archivos del mismo sitio.

Listas

Lista desordenada

Se crea con y cada elemento se define con .

```
<ul>
```

```
<li>HTML</li>
```

```
<li>CSS</li>
```

```
<li>JavaScript</li>
```

```
</ul>
```

Lista ordenada

Se crea con .

```
<ol>
```

```
<li>Abrir el editor</li>
```

Crear el archivo HTML

Escribir la estructura base

Las listas ordenadas son útiles cuando el orden de los ítems importa.

Contenedores genéricos: div y span

Existen elementos que no describen por sí mismos un tipo específico de contenido, pero sirven para agrupar partes del documento.

<div>

Es un contenedor genérico de bloque.

<div>

<h2>Noticias</h2>

<p>Contenido de la sección.</p>

</div>

Es un contenedor genérico en línea.

<p>El resultado fue aprobado.</p>

En HTML moderno conviene usar primero elementos semánticos cuando existan, y dejar div y span para los casos en que no haya una etiqueta más expresiva.

HTML semántico

Decimos que un HTML es **semántico** cuando utiliza etiquetas que describen el rol del contenido.

En lugar de agrupar todo con div, HTML ofrece etiquetas como:

- <header>
- <nav>
- <main>
- <section>
- <article>
- <aside>
- <footer>

Por ejemplo:

```
<body>

<header>

<h1>Portal de noticias</h1>

</header>

<main>

<article>

<h2>Título de la noticia</h2>

<p>Contenido de la noticia.</p>

</article>

</main>

<footer>

<p>Copyright 2026</p>

</footer>

</body>
```

Usar etiquetas semánticas mejora:

- la legibilidad del código
- la accesibilidad
- la interpretación del documento por parte de buscadores y herramientas

Tablas

Las tablas sirven para representar **datos tabulares**, es decir, información organizada en filas y columnas.

```
<table>

<thead>

<tr>

<th>Nombre</th>

<th>Nota</th>

</tr>
```

```
</thead>

<tbody>

<tr>

<td>Ana</td>

<td>9</td>

</tr>

<tr>

<td>Pedro</td>

<td>7</td>

</tr>

</tbody>

</table>
```

Etiquetas principales:

- <table>: tabla completa
- <thead>: encabezado de la tabla
- <tbody>: cuerpo de la tabla
- <tr>: fila
- <th>: celda de encabezado
- <td>: celda de datos

No deben usarse tablas para maquetar páginas. Su uso correcto es representar datos.

Formularios

Los formularios permiten ingresar y enviar datos.

Un ejemplo básico:

```
<form>

<label for="nombre">Nombre</label>

<input id="nombre" type="text" name="nombre" />

<label for="email">Email</label>
```

```
<input id="email" type="email" name="email" />
```

```
<button type="submit">Enviar</button>
```

```
</form>
```

Elementos frecuentes en formularios:

- `<form>`: formulario completo
- `<label>`: texto asociado a un campo
- `<input>`: campo de entrada
- `<textarea>`: área de texto multilínea
- `<select>`: lista desplegable
- `<button>`: botón

Aunque más adelante se verá en mayor profundidad, es importante entender que los formularios son una de las principales formas de interacción entre la persona usuaria y una aplicación web.

Comentarios

Los comentarios permiten dejar anotaciones en el código que no se muestran en la página.

```
<!-- Inicio del encabezado -->
```

```
<header>
```

```
<h1>Mi sitio</h1>
```

```
</header>
```

```
<!-- Fin del encabezado -->
```

Pueden servir para:

- organizar visualmente el archivo
- explicar decisiones puntuales
- marcar secciones durante el desarrollo

No conviene abusar de ellos si el código ya es suficientemente claro.

Buenas prácticas iniciales

Al empezar a escribir HTML conviene adoptar algunas costumbres desde el primer momento:

1. Usar una estructura base completa con `<!DOCTYPE html>`, `<html>`, `<head>` y `<body>`
2. Escribir las etiquetas en minúscula
3. Indentar el código de forma consistente
4. Cerrar correctamente los elementos
5. Usar nombres de archivo simples y descriptivos
6. Preferir etiquetas semánticas cuando existan
7. Escribir texto alternativo útil en las imágenes
8. No usar HTML para resolver problemas de diseño que corresponden a CSS

Resumen

En este primer contacto con HTML conviene quedarse con estas ideas:

- HTML define la estructura de una página web
- trabaja con elementos, etiquetas y atributos
- los documentos tienen una estructura general común
- los elementos pueden anidarse
- existen etiquetas para títulos, párrafos, enlaces, imágenes, listas, tablas y formularios
- HTML semántico ayuda a escribir documentos más claros y accesibles

Aprender HTML no consiste solo en memorizar etiquetas, sino en comprender cómo representar información de manera ordenada y con significado. Esa base será clave para todo lo que siga en desarrollo web.

¿Qué es CSS?

CSS significa **Cascading Style Sheets**. Es el lenguaje que se utiliza para describir la **presentación visual** de un documento HTML.

Mientras que HTML define la estructura y el significado del contenido, CSS indica cómo debe verse ese contenido en pantalla.

Por ejemplo, con CSS se puede decidir:

- qué color tendrá un texto
- qué tipografía se utilizará
- cuánto espacio habrá entre elementos
- si un bloque tendrá borde, fondo o sombra
- cómo se distribuirán los elementos en la página

¿Por qué conviene separar HTML y CSS?

Separar el contenido de la presentación trae varias ventajas importantes:

- mejora el mantenimiento del proyecto
- evita repetir estilos en muchos archivos
- permite reutilizar una misma hoja de estilos en varias páginas
- facilita cambiar el diseño sin tocar la estructura HTML
- ayuda a mantener el código más ordenado y legible

En un proyecto pequeño puede parecer más rápido escribir estilos directamente en cada elemento, pero en cuanto el sitio crece esa estrategia se vuelve difícil de mantener.

Primer ejemplo

Supongamos el siguiente documento HTML:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8" />
<title>Ejemplo CSS</title>
</head>
<body>
<h1>Programación III</h1>
<p>Primer ejemplo con estilos.</p>
</body>
</html>
```

Si agregamos la siguiente hoja de estilo:

```
body {
font-family: Arial, sans-serif;
background-color: #f4f4f4;
}
h1 {
color: #1d4ed8;
}
p {
color: #333333;
}
```

el contenido seguirá siendo el mismo, pero cambiará su apariencia.

La regla CSS

Una regla CSS está formada por un **selector** y un **bloque de declaraciones**.

En una regla como esta:

```
p{  
color: blue;  
font-size: 18px;  
}
```

- p es el **selector**
- color y font-size son **propiedades**
- blue y 18px son **valores**

Cada declaración termina en punto y coma, y el conjunto de declaraciones se escribe entre llaves.

Comentarios en CSS

Los comentarios permiten dejar aclaraciones dentro del archivo sin afectar el resultado visual.

```
/* Estilo principal del encabezado */  
  
h1 {  
color: navy;  
}
```

Formas de conectar CSS con HTML

Existen tres maneras habituales de aplicar CSS a una página.

1. En línea
2. Interno o integrado

3. Externo

En línea

Se utiliza el atributo style dentro de una etiqueta HTML.

```
<p style="color: red">Un texto</p>
```

Este enfoque sirve para casos puntuales, pero tiene varias desventajas:

- mezcla estructura con presentación
- obliga a repetir estilos
- dificulta el mantenimiento
- no escala bien en proyectos medianos o grandes

Por eso, en general **no se recomienda como estrategia principal**.

- ¿Cómo se podrían modificar todos los párrafos del archivo sin repetir style?

Interno o integrado

Se escribe CSS dentro de una etiqueta <style> ubicada en el <head>.

```
<head>
```

```
<style>
```

```
p{
```

```
color: blue;
```

```
font-size: 12px;
```

```
}
```

```
</style>
```

```
</head>
```

Este enfoque permite definir estilos para toda una página, pero sigue teniendo un límite: los estilos quedan atados a un único archivo HTML.

- ¿Qué pasa si hay que aplicar esos mismos estilos en varias páginas?

Externo

La opción más usada es guardar las reglas CSS en un archivo separado y enlazarlo desde HTML con <link>.

```
<!DOCTYPE html>
```

```
<html lang="es">

<head>

<meta charset="UTF-8" />

<title>Ejemplo</title>

<link rel="stylesheet" href="estilo.css" />

</head>

<body>

...

</body>

</html>
```

Por ejemplo, el archivo estilo.css podría contener:

```
body {

background-color: white;

color: #222;

}
```

¿Por qué se prefieren los archivos CSS externos?

- permiten reutilizar estilos en varias páginas
- reducen repetición de código
- facilitan el mantenimiento
- ayudan a organizar mejor el proyecto
- el navegador puede almacenar la hoja de estilos en caché

Sintaxis básica

La unidad fundamental de CSS es la regla:

```
selector {

propiedad: valor;

propiedad: valor;
```

```
}
```

Ejemplo:

```
h1 {  
  color: green;  
  text-align: center;  
}
```

Selectores CSS

Los selectores indican **a qué elementos HTML** se aplicará una regla.

Algunos selectores básicos son:

- selector de elemento
- selector de clase
- selector de id
- selector universal
- selector descendente
- agrupación de selectores

Selector de elemento

Selecciona todos los elementos con un nombre determinado.

```
p {  
  color: #444;  
}  
  
body {  
  background-color: #fafafa;  
}
```

Si el documento tiene diez párrafos, la regla `p { ... }` afectará a los diez.

Selector de id

Selecciona el elemento que tiene un atributo id específico.

```
<h1 id="encabezado">Este es un encabezado</h1>
```

```
<strong id="nombre">Este es un nombre</strong>
```

```
#encabezado {
```

```
color: yellow;
```

```
}
```

```
#nombre {
```

```
background-color: red;
```

```
}
```

Un id debería ser **único dentro del documento**. Se usa cuando un elemento tiene una identidad particular.

Selector de clase

Selecciona todos los elementos que comparten una clase.

```
<h1 class="titulo">Este es un título</h1>
```

```
<p class="direccion">Esta es una dirección</p>
```

```
.titulo {
```

```
font-size: 36px;
```

```
}
```

```
.direccion {
```

```
font-style: italic;
```

```
}
```

Las clases son muy importantes porque permiten **reutilizar estilos**.

Clase aplicada a un tipo de elemento

También se puede indicar que una clase afecte solo a cierto tipo de elemento.

```
<h1 class="titulo">Este es un título</h1>
```

```
<p class="titulo">Este texto también usa la clase titulo</p>
```

```
h1.titulo {  
  
font-size: 36px;  
  
}
```

En este caso, la regla afecta únicamente a los <h1> que tengan la clase titulo, no a todos los elementos con esa clase.

Selector universal

El selector * selecciona todos los elementos.

```
* {  
  
font-size: 16px;  
  
}
```

También puede combinarse con otros selectores:

```
section * {  
  
color: #333;  
  
}
```

Esta regla afecta a todos los elementos que estén dentro de un <section>.

Selector descendente

Permite seleccionar elementos contenidos dentro de otros elementos.

<p>Esta es una palabra destacada dentro de un párrafo.</p>

```
p strong {  
  
background-color: green;  
  
}
```

La regla no selecciona todos los del documento, sino solo los que están dentro de un <p>.

Agrupación de selectores

Si varios elementos deben compartir el mismo estilo, se pueden agrupar separándolos con comas.

p,


```
h1,  
h2 {  
  color: #1f2937;  
}
```

Esto evita duplicar reglas.

Más de una clase en un elemento

Un mismo elemento puede tener varias clases.

```
<p class="texto importante">Atención: este mensaje es importante.</p>
```

```
.texto {  
  font-family: Arial, sans-serif;  
}  
.importante {  
  color: red;  
  font-weight: bold;  
}
```

Esto permite combinar estilos de forma flexible.

Pseudoclases básicas

Las pseudoclases permiten aplicar estilos según el **estado** de un elemento.

Por ejemplo:

```
a:hover {  
  color: orange;  
}  
input:focus {  
  border-color: blue;  
}
```

- `:hover` se activa cuando el puntero pasa sobre el elemento
- `:focus` se activa cuando el elemento recibe el foco

Son muy usadas en enlaces, botones, formularios y menús.

Cascada, herencia y prioridad

La palabra *cascading* en CSS no es decorativa: describe cómo el navegador decide qué reglas aplicar cuando varias afectan al mismo elemento.

Para entender CSS con claridad conviene distinguir tres ideas:

1. cascada
2. herencia
3. especificidad

Cascada

Puede ocurrir que varias reglas apunten al mismo elemento:

```
p{  
color: blue;  
}  
.destacado {  
color: red;  
}  
#principal {  
color: green;  
}
```

Si un párrafo tiene clase destacado e id principal, varias reglas competirán por definir su color. El navegador debe resolver ese conflicto.

Herencia

Algunas propiedades se heredan automáticamente desde el elemento padre hacia sus hijos. Por ejemplo, el color y parte de las propiedades tipográficas.

```
body {
```

```
color: #222;  
  
font-family: Arial, sans-serif;  
  
}
```

Si no se define otra cosa, muchos elementos dentro del body heredarán esos valores.

No todas las propiedades se heredan. Por ejemplo, márgenes, bordes y anchos no suelen heredarse.

Prioridad general

De manera simplificada, el navegador asigna prioridad según el tipo de selector:

1. estilos en línea
2. selectores por id
3. selectores por clase, atributo o pseudoclase
4. selectores por elemento

Prioridad por especificidad

Cuanto más específica es una regla, mayor peso tiene.

```
.main .sale .clearance p {  
  
color: red;  
  
}  
  
.header .title p {  
  
color: green;  
  
}  
  
.footer p {  
  
color: blue;  
  
}
```

En igualdad de condiciones, la regla más específica tiene prioridad.

Prioridad por posición

Si dos reglas tienen la misma especificidad, gana la que aparece más abajo en el archivo.

```
a {
```

```
background-color: red;
```

```
}
```

```
a{
```

```
background-color: yellow;
```

```
}
```

```
a{
```

```
background-color: black;
```

```
}
```

El enlace terminará con fondo negro.

Ejemplo combinado

```
p{
```

```
color: blue;
```

```
font-family: Helvetica, sans-serif;
```

```
}
```

```
.rojo{
```

```
color: red;
```

```
}
```

```
#especial{
```

```
font-family: Arial, sans-serif;
```

```
}
```

```
<p>Párrafo normal</p>
```

```
<p class="rojo">Párrafo rojo</p>
```

```
<p class="rojo" id="especial">Párrafo rojo con otra fuente</p>
```

Valores CSS

Las propiedades CSS aceptan distintos tipos de valores. Dos grupos muy importantes al comenzar son:

- colores
- longitudes y porcentajes

Colores

El navegador admite varias formas de representar colores.

- nombre del color, por ejemplo red
- hexadecimal, por ejemplo #ff0000
- RGB, por ejemplo rgb(255, 0, 0)
- RGBA, por ejemplo rgba(255, 0, 0, 0.5)
- HSL, por ejemplo hsl(0, 100%, 50%)

Ejemplo:

```
h1 {  
  color: #0f172a;  
}  
  
body {  
  background-color: rgb(240, 240, 240);  
}
```

En la práctica, los formatos hexadecimal y RGB/RGBA son muy comunes.

Longitudes y porcentajes

CSS utiliza muchas unidades. Algunas de las más frecuentes son:

- px: píxeles
- %: porcentaje
- em: relativo al tamaño de fuente del elemento
- rem: relativo al tamaño de fuente del elemento raíz
- pt: puntos, más común en contextos de impresión

Ejemplos:

```
h1 {  
font-size: 32px;  
}  
.contenedor {  
width: 80%;  
}  
p {  
line-height: 1.5;  
}
```

Unidades absolutas y relativas

Conviene distinguir entre dos grandes grupos:

- **absolutas:** como px, cm, mm, in
- **relativas:** como %, em, rem

En interfaces web, las unidades relativas suelen aportar más flexibilidad, sobre todo cuando la página debe adaptarse a distintos tamaños de pantalla.

El valor cero

Cuando el valor es cero, no hace falta indicar la unidad:

```
margin: 0;  
padding: 0;  
border: 0;
```

Propiedades CSS frecuentes

Al comenzar, conviene reconocer algunas propiedades muy usadas.

Color y fondo

La propiedad color cambia el color del texto:

```
p {  
color: yellow;
```

```
}
```

La propiedad background-color cambia el color de fondo:

```
body{
```

```
background-color: red;
```

```
}
```

También existen propiedades como:

- background-image
- background-repeat
- background-position

pero para una primera aproximación suele alcanzar con background-color.

Texto y tipografía

Se puede modificar el aspecto del texto con distintas propiedades.

font-family

Indica la familia tipográfica que se intentará usar.

```
p{
```

```
font-family: Arial, Helvetica, sans-serif;
```

```
}
```

El navegador intentará usar la primera fuente disponible. Si no la encuentra, probará con la siguiente.

Si el nombre de una fuente tiene espacios, debe escribirse entre comillas:

```
h1{
```

```
font-family: "Times New Roman", serif;
```

```
}
```

Otras propiedades tipográficas

- font-size: tamaño del texto
- font-weight: grosor de la fuente
- font-style: estilo, por ejemplo cursiva
- text-decoration: subrayado, tachado, etc.

- text-transform: mayúsculas, minúsculas, capitalización

Ejemplo:

```
h2 {  
font-size: 24px;  
font-weight: bold;  
text-transform: uppercase;  
}
```

Espaciado y alineación del texto

Otras propiedades útiles son:

- letter-spacing: espacio entre letras
- word-spacing: espacio entre palabras
- line-height: altura de línea
- text-indent: sangría de la primera línea
- text-align: alineación horizontal del texto

Ejemplo:

```
p {  
line-height: 1.6;  
text-align: justify;  
}
```

Buenas prácticas iniciales

Al comenzar a escribir CSS, conviene adoptar algunas costumbres sanas:

- preferir hojas de estilo externas
- usar nombres de clases claros y descriptivos
- evitar estilos en línea salvo casos excepcionales
- agrupar reglas relacionadas
- no repetir estilos innecesariamente

- mantener una indentación consistente

Por ejemplo, una clase como `.boton-principal` comunica mejor su intención que un nombre poco expresivo como `.azul-grande`.

Resumen

CSS permite controlar la apariencia visual de un documento HTML.

En esta introducción aparecieron varios conceptos fundamentales:

- qué rol cumple CSS en una página web
- cómo se escribe una regla
- cómo conectar CSS con HTML
- qué tipos de selectores existen
- cómo funcionan la cascada, la herencia y la especificidad
- qué valores y unidades son frecuentes
- qué propiedades básicas conviene conocer

A medida que se avance en desarrollo web, CSS incorporará nuevos temas como modelo de caja, display, Flexbox y Grid.

Introducción

En HTML, no todos los elementos se comportan de la misma manera cuando el navegador los muestra en pantalla. Algunos elementos se distribuyen ocupando una nueva línea completa, mientras que otros se ubican uno al lado del otro dentro de la misma línea de texto.

Esta diferencia es fundamental para comprender como se construye la estructura visual de una página web y como CSS puede modificar ese comportamiento.

De forma simplificada, muchos elementos HTML se presentan por defecto como:

- **Elementos de bloque**
- **Elementos en línea**

Mas adelante veremos que este comportamiento puede modificarse con la propiedad CSS display, pero primero conviene entender el comportamiento predeterminado de cada tipo.

Elementos de bloque

Un **elemento de bloque** suele tener las siguientes características:

- comienza en una nueva linea
- ocupa, por defecto, todo el ancho disponible de su contenedor
- se apila verticalmente con respecto a otros elementos de bloque
- suele utilizarse para organizar grandes secciones del contenido

Algunos ejemplos frecuentes de elementos de bloque son:

<div>, <p>, <h1> a <h6>, <section>, <article>, <header>, <footer>, , , , <table>, <form>, <pre>

Por ejemplo, si se escriben varios párrafos seguidos, cada uno aparecerá debajo del anterior:

<p>Primer párrafo</p>

<p>Segundo párrafo</p>

<p>Tercer párrafo</p>

Aunque el contenido de cada párrafo sea corto, cada <p> se mostrará en una línea nueva porque su comportamiento predeterminado es de bloque.

Elementos en linea

Un **elemento en linea** se caracteriza por:

- no comenzar en una nueva linea
- ocupar solo el ancho necesario para mostrar su contenido
- convivir con otros elementos dentro de la misma linea
- usarse normalmente para marcar o modificar fragmentos pequeños de contenido

Algunos ejemplos comunes de elementos en línea son:

, <a>, , , , <i>, <small>, , <label>

Por ejemplo:

<p>

Este texto contiene una parte importante y un

enlace.

</p>

En este caso, tanto como <a> permanecen dentro de la misma línea del párrafo, porque son elementos en línea.

Comparación entre bloque y línea

La diferencia principal entre ambos tipos no está en su significado semántico, sino en **como participan en el flujo normal del documento**.

Tipo de elemento	Comienza en nueva línea	Ocupa todo el ancho disponible	Uso habitual
Bloque	Si	Si, por defecto	Estructurar secciones o grupos grandes
En línea	No	No	Marcar o estilizar partes de un texto

Es importante notar que "de bloque" y "en línea" describen el **comportamiento visual inicial**, no la importancia del contenido.

Ejemplo comparativo

<h1>Titulo principal</h1>

<p>

Este es un párrafo con una palabra destacada y un

`<a href="#">enlace</a>.`

`</p>`

`<p>Otro párrafo.</p>`

En este ejemplo:

- `<h1>` es de bloque, por lo tanto aparece en su propia línea
- cada `<p>` también es de bloque
- `<span>` y `<a>` son elementos en línea, por eso quedan dentro del texto del párrafo

El elemento `div`

El elemento `<div>` es uno de los contenedores más utilizados en HTML.

Por defecto, `<div>` es un **elemento de bloque**, por lo que resulta útil para:

- agrupar contenido relacionado
- dividir una página en secciones
- aplicar estilos CSS a un conjunto de elementos
- organizar la estructura de una interfaz

`<div>` no aporta un significado semántico especial. Es un contenedor **genérico**.

Por eso, se lo usa cuando no existe un elemento semántico más adecuado, como `<section>`, `<article>`, `<nav>` o `<header>`.

Ejemplo:

`<div style="background-color: black; color: white; padding: 20px">`

`<h2>Londres</h2>`

`<p>`

Londres es la capital de Inglaterra. Es la ciudad mas poblada del Reino

Unido, con un area metropolitana de mas de 13 millones de habitantes.

`</p>`

`</div>`

En este caso, el `<div>` funciona como una caja que agrupa un título y un párrafo para aplicarles una misma presentación visual.

El elemento `span`

El elemento `<span>` es el equivalente en línea de `<div>`.

Por defecto, `<span>` es un **elemento en línea**. Se lo utiliza para:

- agrupar una parte del texto
- aplicar estilos a palabras o fragmentos concretos
- agregar clases o identificadores a contenido pequeño

Tampoco tiene significado semántico propio: es un contenedor genérico, pero en línea.

Ejemplo:

```
<h1>Mi encabezado <span style="color: red">importante</span></h1>
```

En este caso, solo la palabra incluida dentro de `<span>` cambiara de color, sin romper la línea del encabezado.

Elementos de agrupación

En términos prácticos, `<div>` y `<span>` suelen considerarse **elementos de agrupación** porque permiten reunir contenido para manipularlo con CSS o con JavaScript.

- `<div>` agrupa contenido a nivel de bloque
- `<span>` agrupa contenido a nivel de línea

Por ejemplo:

```
<body>
```

```
<div id="encabezado">
```

```
<h1>Titulo de pagina</h1>
```

```
</div>
```

```
<div id="contenido">
```

```
<p>Contenido principal</p>
</div>
<div id="barra-lateral">
<p>Algo <span class="resaltado">resaltado</span> en la barra lateral</p>
</div>
<div id="pie">
<p>Derecho de autor</p>
</div>
</body>
```

Aquí, los distintos `<div>` separan grandes zonas de la pagina, mientras que `<span>` se utiliza para marcar una parte puntual del texto.

Relación con CSS y la propiedad display

Aunque un elemento tenga un comportamiento predeterminado, CSS puede modificarlo mediante la propiedad `display`.

Por ejemplo:

```
span {
display: block;
}
div {
display: inline;
}
```

Con esas reglas:

- un `<span>` pasaría a comportarse como un elemento de bloque
- un `<div>` pasaría a comportarse como un elemento en línea

Esto demuestra que el comportamiento visual puede cambiar, aunque el elemento HTML siga siendo el mismo.

Sin embargo, cambiar display **no modifica el significado semántico** del elemento. Un `<div>` sigue siendo un contenedor genérico, aunque se vea en línea, y un `<span>` sigue siendo un contenedor genérico, aunque se muestre como bloque.

Una aclaración importante

En los cursos iniciales suele hablarse de "elementos de bloque" y "elementos en línea" como si fueran categorías rígidas. Esa simplificación es útil para aprender, pero en la práctica hay más posibilidades.

CSS ofrece otros valores de display, por ejemplo:

- inline-block
- flex
- grid
- none

Por eso, conviene entender esta clasificación como un **punto de partida** para comprender el flujo del documento y no como una lista cerrada de comportamientos posibles.

Errores conceptuales frecuentes

1. Pensar que un elemento "de bloque" siempre ocupa toda la pantalla. En realidad, ocupa todo el ancho disponible dentro de su contenedor.
2. Creer que un elemento en línea no puede recibir estilos. Si puede recibirlos, aunque ciertas propiedades no se comportan igual que en los elementos de bloque.
3. Confundir comportamiento visual con significado semántico. Una cosa es como se muestra un elemento y otra distinta es que representa dentro del documento.
4. Usar `<div>` para todo. En HTML moderno conviene preferir elementos semánticos cuando existen opciones más expresivas.

Síntesis

Los elementos de bloque y los elementos en línea representan dos formas básicas de organizar el contenido en HTML.

- los elementos de bloque estructuran secciones más amplias del documento
- los elementos en línea trabajan sobre fragmentos pequeños de contenido
- `<div>` y `<span>` son contenedores genéricos muy usados
- CSS puede alterar el comportamiento visual mediante `display`

Comprender esta diferencia permite escribir HTML más claro y preparar el terreno para temas posteriores, como el modelo de caja, el posicionamiento y los sistemas de distribución en CSS.

Introducción

Uno de los conceptos más importantes de CSS es el **modelo de caja** o **box model**.

Cuando el navegador representa muchos elementos HTML, los interpreta como si fueran **cajas rectangulares**. Esa idea no siempre es visible a simple vista, pero resulta fundamental para entender:

- ¿por qué un elemento ocupa cierto espacio?
- ¿por qué dos elementos quedan separados?
- ¿cómo se calculan anchos y altos?
- ¿cómo se construye la disposición visual de una página?

Comprender el modelo de caja evita muchos errores comunes al trabajar con espaciado, bordes, tamaños y alineación.

Las cuatro áreas de una caja

De manera simplificada, cada caja puede pensarse como formada por cuatro zonas:

1. contenido
2. padding
3. borde
4. margen

Por ejemplo:

```
.tarjeta {  
width: 300px;  
padding: 16px;  
border: 1px solid #cccccc;  
margin: 20px;  
}
```

En ese caso, el elemento tiene un área de contenido, un espacio interno, un borde visible y una separación externa con respecto a otros elementos.

Área de contenido

El **contenido** es la parte central de la caja.

Allí se muestran:

- el texto
- las imágenes
- otros elementos HTML anidados

Cuando se define width o height, en el modelo estándar de CSS esos valores se aplican, por defecto, al **área de contenido**.

Ejemplo:

```
.caja {  
width: 300px;  
height: 120px;
```

```
}
```

Esto no significa necesariamente que la caja completa mida 300 píxeles de ancho, porque luego pueden sumarse padding y border.

Padding

El **padding** es el espacio interno entre el contenido y el borde.

Sirve para que el contenido no quede "pegado" al borde del elemento.

```
.boton {  
padding: 10px 16px;  
}
```

Si un botón tiene padding, el texto interior gana aire y la caja se ve más clara y cómoda para leer.

El padding:

- aumenta el espacio interior
- se encuentra dentro del borde
- forma parte del tamaño visual del elemento

Border

El **borde** es la línea que rodea al contenido y al padding.

Se puede definir su:

- grosor
- estilo
- color

```
.caja {  
border: 2px solid black;  
}
```

El borde también ocupa espacio y, por lo tanto, influye en el tamaño final que se ve en pantalla.

Margin

El **margin** es el espacio externo que separa una caja de otras cajas.

```
.bloque {  
margin: 24px;  
}
```

A diferencia del padding:

- el padding está dentro de la caja
- el margin está fuera de la caja

Esta diferencia es clave. Si se quiere separar el contenido del borde, se usa padding. Si se quiere separar un elemento de otros elementos, se usa margin.

Resumen visual del modelo

Puede pensarse así, desde adentro hacia afuera:

1. contenido
2. padding
3. borde
4. margin

O, expresado como idea:

el contenido vive dentro de la caja, el padding le da aire, el borde la delimita y el margin la separa de las demás

Ancho y alto en el modelo estándar

Una de las confusiones más frecuentes aparece con width y height.

En el modelo estándar de CSS:

- width define el ancho del contenido
- height define el alto del contenido

Si además se agregan padding y border, el tamaño total visible será mayor.

Por ejemplo:

```
.panel {  
width: 300px;  
padding: 20px;  
border: 2px solid gray;  
}
```

En este caso, el ancho total visible será:

- 300px de contenido
- 20px de padding izquierdo
- 20px de padding derecho
- 2px de borde izquierdo
- 2px de borde derecho

Total: **344px**

Este detalle explica por qué a veces una caja "no entra" en el espacio esperado, aunque el ancho definido parezca correcto.

La propiedad box-sizing

Para simplificar el cálculo del tamaño real de una caja, CSS ofrece la propiedad box-sizing.

content-box

Es el valor predeterminado.

```
.caja {  
box-sizing: content-box;
```

```
}
```

Con este valor:

- width y height afectan solo al contenido
- padding y border se suman por fuera

border-box

Es muy usado en proyectos reales porque vuelve más predecible el cálculo del tamaño.

```
.caja {  
  
box-sizing: border-box;  
  
width: 300px;  
  
padding: 20px;  
  
border: 2px solid gray;  
  
}
```

Con border-box, el ancho total de la caja sigue siendo de 300px, y dentro de ese valor quedan incluidos el contenido, el padding y el borde.

Por eso, en muchos proyectos se utiliza una regla general como esta:

```
* {  
  
box-sizing: border-box;  
  
}
```

No cambia el significado del modelo de caja, pero sí cambia la forma en que se calcula el tamaño declarado.

Propiedades más usadas del modelo de caja

width y height

Definen el ancho y el alto del contenido, salvo que se esté usando box-sizing: border-box.

```
img {  
  
width: 200px;
```

```
height: 150px;
```

```
}
```

padding

Define espacio interno.

```
button {
```

```
padding: 10px 16px;
```

```
}
```

border

Permite establecer grosor, estilo y color del borde.

```
.caja {
```

```
border: 2px solid black;
```

```
}
```

margin

Define espacio externo.

```
.bloque {
```

```
margin: 24px;
```

```
}
```

Propiedades abreviadas

CSS permite escribir estas propiedades de forma abreviada.

Por ejemplo:

```
.caja {
```

```
margin: 10px 20px 30px 40px;
```

```
}
```

Ese orden significa:

1. arriba
2. derecha

3. abajo

4. izquierda

También se pueden usar formas más cortas:

```
.caja {  
padding: 16px;  
margin: 10px 20px;  
}
```

En ese caso:

- padding: 16px aplica el mismo valor a los cuatro lados
- margin: 10px 20px aplica 10px arriba y abajo, y 20px a izquierda y derecha

Relación con los elementos de bloque y en línea

El modelo de caja se entiende mejor cuando se lo relaciona con el tipo de elemento.

Los elementos de bloque, como <div> o <p>, suelen aceptar de manera natural:

- ancho
- alto
- márgenes verticales
- padding
- bordes

Los elementos en línea, como o <a>, también participan del modelo de caja, pero no siempre se comportan de la misma manera respecto del ancho, el alto o la distribución vertical.

Por eso, al estudiar el modelo de caja conviene conectarlo con la propiedad display.

Colapso de márgenes

Otro detalle importante es que los márgenes verticales de algunos elementos de bloque pueden **colapsar**.

Por ejemplo, si un elemento tiene margin-bottom: 20px y el siguiente tiene margin-top: 30px, no siempre se suman para dar 50px. En muchos casos, el navegador usa solo el valor mayor.

Este comportamiento puede sorprender al principio, porque no ocurre con el padding ni con los bordes.

No es necesario dominar todos los casos desde el comienzo, pero sí saber que el colapso de márgenes existe.

Errores frecuentes

1. Pensar que width representa siempre el ancho total visible del elemento. En el modelo estándar, no es así.
2. Usar margin cuando en realidad se necesita padding.
3. Olvidar que el borde también ocupa espacio.
4. Suponer que todos los elementos responden igual a width y height.
5. No considerar box-sizing, lo que suele generar cálculos inesperados.

Síntesis

El modelo de caja describe cómo CSS representa muchos elementos HTML en forma de cajas rectangulares.

Cada caja está compuesta por:

1. contenido
2. padding
3. borde
4. margen

Entender este modelo permite:

- calcular tamaños con mayor precisión
- controlar mejor la separación entre elementos
- diseñar interfaces más prolijas y predecibles
- evitar errores comunes de espaciado

Es un tema central para todo lo que sigue en CSS: distribución, alineación, posicionamiento y diseño de interfaces.

¿Qué es JavaScript?

JavaScript es un lenguaje de programación que se usa principalmente para agregar **comportamiento** a las aplicaciones web.

En una página o aplicación web suele aparecer junto con:

- **HTML**, que define la estructura
- **CSS**, que define la presentación visual
- **JavaScript**, que define la lógica y el comportamiento

JavaScript permite:

- guardar datos en variables
- tomar decisiones
- repetir acciones
- agrupar instrucciones en funciones
- trabajar con colecciones de datos
- dividir código en módulos

Aunque muchas veces se lo asocia con el navegador, JavaScript también puede ejecutarse fuera de él, por ejemplo en servidores con **Node.js**.

¿Por qué es central en desarrollo web?

Porque permite programar la lógica de una aplicación:

- validar datos

- transformar información
- calcular resultados
- organizar código reutilizable
- conectar distintas partes del sistema

En este apunte nos enfocaremos en los fundamentos del lenguaje. No veremos **DOM** ni **eventos**, ya que esos temas se tratarán por separado.

Formas de usar JavaScript en HTML

Existen dos formas habituales de incluir JavaScript en una página:

1. dentro del mismo archivo HTML con `<script>`
2. en un archivo .js separado

JavaScript dentro de `<script>`

Se puede escribir código JavaScript directamente dentro del documento HTML:

```
<!DOCTYPE html>

<html lang="es">

<head>

<meta charset="UTF-8" />

<title>Ejemplo</title>

</head>

<body>

<script>

console.log("Hola desde JavaScript");

let nombre = "Ana";

console.log("Bienvenida, " + nombre);

</script>

</body>

</html>
```

Este enfoque puede ser útil para ejemplos pequeños o pruebas rápidas.

Sin embargo, tiene desventajas:

- mezcla estructura con lógica
- dificulta reutilizar código
- complica el mantenimiento cuando el proyecto crece

JavaScript en archivo separado

La forma más recomendada en proyectos reales es escribir el código en un archivo aparte y enlazarlo desde HTML.

Por ejemplo, si tenemos este archivo app.js:

```
console.log("JavaScript cargado desde archivo externo");
```

```
let curso = "Programación III";
```

```
console.log(curso);
```

podemos conectarlo así:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<title>Ejemplo</title>
```

```
</head>
```

```
<body>
```

```
<script src="app.js"></script>
```

```
</body>
```

```
</html>
```

¿Por qué conviene usar archivos separados?

- separan contenido y lógica
- facilitan mantenimiento
- permiten reutilizar código
- mejoran organización del proyecto

- resultan más cómodos para trabajar en equipo

Variables y tipos de datos

Una **variable** es un nombre que referencia un valor.

```
let nombre = "Lucía";
```

```
let edad = 20;
```

```
let activo = true;
```

En JavaScript los valores pueden cambiar durante la ejecución del programa, y el lenguaje determina el tipo de dato en tiempo de ejecución.

Tipos de datos básicos

string

Representa texto.

```
let mensaje = "Hola";
```

```
let ciudad = 'Rosario';
```

number

Representa números enteros y decimales.

```
let cantidad = 10;
```

```
let precio = 99.5;
```

boolean

Representa verdadero o falso.

```
let estaAprobado = true;
```

```
let tieneErrores = false;
```

undefined

Indica que una variable existe, pero todavía no tiene valor asignado.

```
let resultado;
```

```
console.log(resultado); // undefined
```

null

Representa ausencia intencional de valor.

```
let usuarioActual = null;
```

object

Agrupar datos relacionados mediante pares clave-valor.

```
let alumno = {  
  nombre: "Marcos",  
  edad: 21  
};
```

Otros tipos

También existen otros tipos menos usados en una introducción inicial, como bigint y symbol.

Declaración de variables: var, let y const

JavaScript ofrece tres formas principales de declarar variables:

```
var materia = "Programación";
```

```
let comision = "A";
```

```
const anio = 2026;
```

var

Es la forma más antigua.

```
var nombre = "Juan";
```

```
var nombre = "Pedro";
```

Características importantes:

- tiene **alcance de función**, no de bloque
- permite redeclarar la misma variable
- puede reasignarse

- su comportamiento puede generar errores difíciles de detectar

Ejemplo:

```
if (true) {  
  var mensaje = "Hola";  
}  
  
console.log(mensaje); // "Hola"
```

Aunque mensaje fue declarada dentro del if, sigue existiendo fuera del bloque.

let

Se usa para variables cuyo valor puede cambiar.

```
let contador = 0;  
  
contador = contador + 1;
```

Características:

- tiene **alcance de bloque**
- no permite redeclarar en el mismo bloque
- sí permite reasignar

```
if (true) {  
  let estado = "activo";  
}  
  
// console.log(estado); // Error: estado no existe fuera del bloque
```

const

Se usa cuando la referencia no debe cambiar.

```
const pi = 3.14159;
```

Características:

- tiene **alcance de bloque**
- no permite redeclarar
- no permite reasignar
- debe inicializarse al declararse

```
const curso = "Programación III";
```

```
// curso = "Otra materia"; // Error
```

Importante sobre const

Que una variable sea const no significa que el contenido del objeto o arreglo sea inmutable. Significa que **la referencia** no puede cambiar.

```
const alumno = {  
  nombre: "Sofía"  
};  
  
alumno.nombre = "Martina"; // válido
```

Lo que no se puede hacer es esto:

```
const alumno = {  
  nombre: "Sofía"  
};  
  
// alumno = { nombre: "Martina" }; // Error
```

Comparativa entre var, let y const

Característica	var	let	const
Alcance	función	bloque	bloque
Redeclaración en mismo bloque	sí	no	no
Reasignación	sí	sí	no
Inicialización obligatoria	no	no	sí
Uso recomendado actual	no	sí	sí

En código moderno, la práctica habitual es:

- usar **const** por defecto

- usar **let** cuando el valor debe cambiar
- evitar **var** salvo que se esté leyendo o manteniendo código antiguo

Condicionales

Los condicionales permiten ejecutar distintos bloques de código según una condición.

if

```
let nota = 8;  
  
if (nota >= 6) {  
  
  console.log("Aprobado");  
  
}
```

if / else

```
let edad = 17;  
  
if (edad >= 18) {  
  
  console.log("Mayor de edad");  
  
} else {  
  
  console.log("Menor de edad");  
  
}
```

if / else if / else

```
let nota = 4;  
  
if (nota >= 9) {  
  
  console.log("Excelente");  
  
} else if (nota >= 6) {  
  
  console.log("Aprobado");  
  
} else {  
  
  console.log("Desaprobado");  
  
}
```


switch

switch es útil cuando se quiere comparar una misma expresión contra varios casos posibles.

```
let dia = 3;

switch (dia) {

  case 1:

    console.log("Lunes");

    break;

  case 2:

    console.log("Martes");

    break;

  case 3:

    console.log("Miércoles");

    break;

  default:

    console.log("Otro día");

}
```

El break evita que la ejecución continúe en los siguientes casos.

Operador ternario

Es una forma breve de expresar una decisión simple.

```
let edad = 20;

let mensaje = edad >= 18 ? "Puede ingresar" : "No puede ingresar";

console.log(mensaje);
```

Suele ser conveniente para decisiones cortas. Si la lógica es más compleja, en general resulta más claro usar if.

Bucles

Los bucles permiten repetir instrucciones.

for

Se usa mucho cuando se conoce de antemano cuántas veces se quiere repetir algo.

```
for (let i = 0; i < 5; i++) {  
  console.log("Vuelta número:", i);  
}
```

Partes del for:

1. inicialización
2. condición
3. actualización

while

Se repite mientras la condición sea verdadera.

```
let contador = 0;  
while (contador < 3) {  
  console.log(contador);  
  contador++;  
}
```

Si la condición nunca deja de cumplirse, se genera un bucle infinito.

do...while

Primero ejecuta el bloque y después evalúa la condición. Por eso, se ejecuta al menos una vez.

```
let numero = 5;  
do {  
  console.log("Valor:", numero);  
  numero++;  
} while (numero < 5);
```

En este caso el bloque se ejecuta una vez, aunque la condición ya sea falsa.

Funciones

Una **función** agrupa instrucciones reutilizables.

```
function saludar() {  
  console.log("Hola");  
}  
  
saludar();
```

Funciones con parámetros

```
function saludarPersona(nombre) {  
  console.log("Hola, " + nombre);  
}  
  
saludarPersona("Carla");
```

Funciones con retorno

```
function sumar(a, b) {  
  return a + b;  
}  
  
let resultado = sumar(10, 5);  
console.log(resultado);
```

Funciones flecha

Las **funciones flecha** son una sintaxis más corta para definir funciones.

```
const saludar = () => {  
  console.log("Hola");  
};
```

Con un parámetro:

```
const cuadrado = numero => {  
  return numero * numero;  
};
```

Si el cuerpo tiene una sola expresión, se puede usar retorno implícito:

```
const duplicar = numero => numero * 2;
```

Diferencias entre función tradicional y función flecha

1. Sintaxis

Función tradicional:

```
function sumar(a, b) {  
  return a + b;  
}
```

Función flecha:

```
const sumar = (a, b) => {  
  return a + b;  
};
```

2. this

Las funciones flecha **no tienen su propio this**. Toman el this del contexto exterior.

Las funciones tradicionales sí pueden tener su propio this, según cómo se las invoque.

Este tema se vuelve especialmente importante en objetos, clases y callbacks.

3. arguments

Las funciones tradicionales disponen del objeto especial arguments.

```
function ejemplo() {  
  console.log(arguments);  
}
```

Las funciones flecha no lo tienen.

4. Uso como constructor

Una función tradicional puede usarse con new en determinados contextos. Una función flecha no.

5. Hoisting

Las declaraciones de función suelen poder invocarse antes de aparecer en el código fuente:

```
saludar();

function saludar() {
  console.log("Hola");
}
```

En cambio, una función flecha asignada a `const` o `let` no puede usarse antes de la declaración:

```
// saludar(); // Error

const saludar = () => {
  console.log("Hola");
};
```

¿Cuál conviene usar?

Depende del caso:

- función tradicional: cuando interesa una definición clásica, hoisting o un comportamiento propio de `this`
- función flecha: cuando se busca sintaxis breve o capturar `this` del contexto exterior

Arreglos

Un **arreglo** almacena varios valores en una misma estructura.

```
let numeros = [10, 20, 30, 40];
let nombres = ["Ana", "Luis", "Mora"];
let mixto = [1, "texto", true];
```

Los elementos se acceden por índice, comenzando en 0.

```
let colores = ["rojo", "verde", "azul"];
console.log(colores[0]); // rojo
console.log(colores[2]); // azul
```

También se pueden modificar:

```
let frutas = ["manzana", "pera", "banana"];
```

```
frutas[1] = "naranja";
```

Recorrer un arreglo

```
let notas = [7, 8, 10];
```

```
for (let i = 0; i < notas.length; i++) {
```

```
  console.log(notas[i]);
```

```
}
```

Arreglos en línea

Muchas veces se crean arreglos directamente en el lugar donde se necesitan:

```
console.log(["HTML", "CSS", "JavaScript"]);
```

También pueden combinarse con objetos:

```
const curso = {
```

```
  nombre: "Programación III",
```

```
  temas: ["Git", "HTML", "CSS", "JavaScript"]
```

```
};
```

Objetos en línea

Un **objeto** permite agrupar datos mediante propiedades.

```
let alumno = {
```

```
  nombre: "Valentina",
```

```
  edad: 22,
```

```
  activo: true
```

```
};
```

Acceso a propiedades:

```
console.log(alumno.nombre);
```

```
console.log(alumno["edad"]);
```

También se pueden modificar o agregar propiedades:

```
alumno.edad = 23;
```

```
alumno.comision = "B";
```

Objetos en línea

Al igual que con los arreglos, un objeto puede definirse directamente en el lugar donde se lo necesita:

```
console.log({  
  
  marca: "Ford",  
  
  modelo: "Fiesta",  
  
  anio: 2018  
  
});
```

Arreglos y objetos combinados

Es muy común trabajar con estructuras combinadas:

```
const alumnos = [  
  
  { nombre: "Ana", nota: 8 },  
  
  { nombre: "Luis", nota: 6 },  
  
  { nombre: "Mora", nota: 10 }  
  
];  
  
console.log(alumnos[0].nombre); // Ana  
  
console.log(alumnos[2].nota); // 10
```

Módulos

Cuando un programa crece, conviene dividirlo en varios archivos. En JavaScript, eso se logra con **módulos**.

Un módulo permite:

- separar responsabilidades
- reutilizar código
- importar solo lo necesario

- evitar mezclar todo en un único archivo

Exportar desde un módulo

Archivo matematica.js:

```
export const PI = 3.14159;  
  
export function sumar(a, b) {  
  
  return a + b;  
  
}
```

Importar desde otro módulo

Archivo app.js:

```
import { PI, sumar } from "./matematica.js";  
  
console.log(PI);  
  
console.log(sumar(2, 3));
```

Conectar un módulo en HTML

Para que el navegador interprete el archivo como módulo, se usa:

```
<script type="module" src="app.js"></script>
```

Algunas características de los módulos

- cada archivo tiene su propio alcance
- para reutilizar algo, debe exportarse
- para usar algo de otro archivo, debe importarse
- type="module" habilita sintaxis import y export

Cierre

En esta introducción vimos los elementos básicos del lenguaje:

- cómo incorporar JavaScript a una página
- cómo declarar variables
- qué tipos de datos básicos existen
- cómo usar condicionales y bucles

- cómo definir funciones
- cómo trabajar con arreglos, objetos y módulos

Estos conceptos forman la base para avanzar luego hacia temas como manipulación del DOM, eventos, asincronía, consumo de APIs y desarrollo de aplicaciones más completas.

Introducción

Los arreglos ocupan un lugar central en JavaScript porque permiten trabajar con colecciones de datos:

- listas de nombres
- notas de estudiantes
- productos
- resultados de búsquedas
- registros traídos desde una API

En un primer momento, muchas operaciones sobre arreglos pueden resolverse con `for` o `while`. Sin embargo, JavaScript ofrece una serie de **métodos de arreglo** que permiten expresar esas operaciones de forma más clara y, muchas veces, más cómoda de leer.

Por ejemplo, con estos métodos podemos:

- recorrer elementos
- transformar valores
- filtrar datos
- obtener un único resultado acumulado
- verificar si existe algún elemento que cumpla una condición

En este apunte veremos métodos como `forEach`, `map`, `reduce`, `some`, `filter` y otros similares de uso frecuente. No veremos **DOM** ni **eventos**, porque pertenecen a otro apunte.

Recordatorio: qué es un arreglo

Un arreglo es una estructura que agrupa varios valores en una sola variable.

```
const notas = [7, 9, 6, 10];
```

```
const nombres = ["Ana", "Luis", "Mora"];
```

Cada elemento tiene una posición, llamada **índice**, que comienza en 0.

```
const colores = ["rojo", "verde", "azul"];
```

```
console.log(colores[0]); // rojo
```

```
console.log(colores[2]); // azul
```

Método y callback

Muchos métodos de arreglos reciben una **función** como argumento. A esa función se la suele llamar **callback**.

Ejemplo general:

```
const numeros = [1, 2, 3];
```

```
numeros.forEach(function (numero) {
```

```
  console.log(numero);
```

```
});
```

En este caso:

- `forEach` es método del arreglo
- función que se pasa como argumento es `callback`
- `callback` se ejecuta una vez por cada elemento

En código moderno suele usarse función flecha:

```
const numeros = [1, 2, 3];
```

```
numeros.forEach(numero => {
```

```
  console.log(numero);
```

```
});
```

¿Todos los métodos devuelven un nuevo arreglo?

No. Esta diferencia es importante.

Algunos métodos:

- **recorren** arreglo sin crear otro
- **devuelven** un nuevo arreglo
- **devuelven** un único valor
- **devuelven** true o false

Por eso conviene entender **qué hace** y **qué devuelve** cada uno.

forEach

¿Para qué sirve?

forEach recorre un arreglo y ejecuta una función por cada elemento.

```
const nombres = ["Ana", "Luis", "Mora"];
nombres.forEach(nombre => {
  console.log("Hola, " + nombre);
});
```

Salida:

Hola, Ana

Hola, Luis

Hola, Mora

Parámetros habituales

Callback de forEach puede recibir:

1. elemento actual
2. índice
3. arreglo completo

```
const materias = ["HTML", "CSS", "JavaScript"];
```

```
materias.forEach((materia, indice) => {  
  console.log(indice + ": " + materia);  
});
```

Importante

forEach **no devuelve un nuevo arreglo útil para encadenar transformaciones.**

Se usa sobre todo para:

- mostrar datos
- ejecutar efectos secundarios
- recorrer elementos sin necesidad de construir otro resultado

```
const numeros = [1, 2, 3];  
const resultado = numeros.forEach(numero => numero * 2);  
console.log(resultado); // undefined
```

map

¿Para qué sirve?

map crea un **nuevo arreglo** a partir de transformar cada elemento del arreglo original.

```
const numeros = [1, 2, 3, 4];  
const dobles = numeros.map(numero => numero * 2);  
console.log(dobles); // [2, 4, 6, 8]
```

Idea clave

- map no modifica arreglo original
- map devuelve nuevo arreglo
- nuevo arreglo tiene misma cantidad de elementos

```
const nombres = ["ana", "luis", "mora"];  
const mayusculas = nombres.map(nombre => nombre.toUpperCase());  
console.log(nombres); // ["ana", "luis", "mora"]
```

```
console.log(mayusculas); // ["ANA", "LUIS", "MORA"]
```

Ejemplo con objetos

```
const alumnos = [  
  { nombre: "Ana", nota: 8 },  
  { nombre: "Luis", nota: 6 },  
  { nombre: "Mora", nota: 10 }  
];  
  
const nombres = alumnos.map(alumno => alumno.nombre);  
  
console.log(nombres); // ["Ana", "Luis", "Mora"]
```

filter

¿Para qué sirve?

filter crea un **nuevo arreglo** con los elementos que cumplen una condición.

```
const numeros = [3, 8, 12, 5, 20];  
  
const mayoresA10 = numeros.filter(numero => numero > 10);  
  
console.log(mayoresA10); // [12, 20]
```

Idea clave

- callback debe devolver true o false
- si devuelve true, elemento queda
- si devuelve false, elemento se descarta

Ejemplo con objetos

```
const productos = [  
  { nombre: "Teclado", stock: 5 },  
  { nombre: "Mouse", stock: 0 },  
  { nombre: "Monitor", stock: 3 }  
];  
  
const disponibles = productos.filter(producto => producto.stock > 0);
```

```
console.log(disponibles);
```

Resultado:

```
[  
  { nombre: "Teclado", stock: 5 },  
  { nombre: "Monitor", stock: 3 }  
]
```

some

¿Para qué sirve?

some verifica si **al menos un elemento** cumple una condición.

Devuelve:

- true si encuentra alguno
- false si no encuentra ninguno

```
const notas = [4, 5, 8, 6];
```

```
const hayNotaAlta = notas.some(nota => nota >= 8);
```

```
console.log(hayNotaAlta); // true
```

Ejemplo

```
const usuarios = [
```

```
  { nombre: "Ana", admin: false },
```

```
  { nombre: "Luis", admin: false },
```

```
  { nombre: "Mora", admin: true }  
];
```

```
const existeAdmin = usuarios.some(usuario => usuario.admin === true);
```

```
console.log(existeAdmin); // true
```

every

every es parecido a some, pero verifica si **todos los elementos** cumplen condición.

```
const numeros = [2, 4, 6, 8];  
  
const todosPares = numeros.every(numero => numero % 2 === 0);  
  
console.log(todosPares); // true
```

Si al menos uno no cumple, devuelve false.

```
const notas = [7, 8, 4, 10];  
  
const todasAprobadas = notas.every(nota => nota >= 6);  
  
console.log(todasAprobadas); // false
```

find

find devuelve **primer elemento** que cumple condición.

Si no encuentra ninguno, devuelve undefined.

```
const productos = [  
  { id: 1, nombre: "Teclado" },  
  { id: 2, nombre: "Mouse" },  
  { id: 3, nombre: "Monitor" }  
];  
  
const producto = productos.find(item => item.id === 2);  
  
console.log(producto); // { id: 2, nombre: "Mouse" }
```

Es útil cuando se espera un solo resultado.

findIndex

findIndex funciona parecido a find, pero devuelve **índice** del primer elemento que cumple condición.

Si no encuentra ninguno, devuelve -1.

```
const nombres = ["Ana", "Luis", "Mora"];

const indice = nombres.findIndex(nombre => nombre === "Luis");

console.log(indice); // 1
```

reduce

¿Para qué sirve?

reduce recorre arreglo y lo **reduce** a un único valor.

Ese valor puede ser:

- suma
- promedio
- objeto
- arreglo
- contador
- texto

Ejemplo simple: suma total

```
const numeros = [10, 20, 30];

const total = numeros.reduce((acumulador, numero) => {

  return acumulador + numero;

}, 0);

console.log(total); // 60
```

Partes importantes

En este ejemplo:

- acumulador guarda resultado parcial
- numero es elemento actual
- 0 es valor inicial del acumulador

Ejemplo: contar aprobados

```
const notas = [7, 4, 9, 5, 6];
```



```
const cantidadAprobados = notas.reduce((acumulador, nota) => {  
  if (nota >= 6) {  
    return acumulador + 1;  
  }  
  return acumulador;  
}, 0);  
console.log(cantidadAprobados); // 3
```

Ejemplo: sumar stock de productos

```
const productos = [  
  { nombre: "Teclado", stock: 5 },  
  { nombre: "Mouse", stock: 3 },  
  { nombre: "Monitor", stock: 2 }  
];  
const stockTotal = productos.reduce((acumulador, producto) => {  
  return acumulador + producto.stock;  
}, 0);  
console.log(stockTotal); // 10
```

includes

includes permite saber si un arreglo contiene cierto valor.

```
const tecnologias = ["HTML", "CSS", "JavaScript"];  
console.log(tecnologias.includes("CSS")); // true  
console.log(tecnologias.includes("React")); // false
```

Es muy útil cuando solo interesa saber si valor está o no está.

indexOf

indexOf devuelve posición de un valor dentro de arreglo.

Si no lo encuentra, devuelve -1.

```
const colores = ["rojo", "verde", "azul"];  
console.log(colores.indexOf("verde")); // 1  
console.log(colores.indexOf("negro")); // -1
```

Hoy muchas veces includes resulta más legible si solo se quiere comprobar existencia.

slice

slice devuelve una **copia parcial** de un arreglo sin modificar original.

```
const numeros = [10, 20, 30, 40, 50];  
const parte = numeros.slice(1, 4);  
console.log(parte); // [20, 30, 40]  
console.log(numeros); // [10, 20, 30, 40, 50]
```

Empieza en índice inicial y toma elementos hasta antes de índice final.

concat

concat une arreglos y devuelve uno nuevo.

```
const grupoA = ["Ana", "Luis"];  
const grupoB = ["Mora", "Juan"];  
const cursoCompleto = grupoA.concat(grupoB);  
console.log(cursoCompleto); // ["Ana", "Luis", "Mora", "Juan"]
```

join

join une elementos de arreglo en un solo string.

```
const palabras = ["JavaScript", "es", "flexible"];
const oracion = palabras.join(" ");
console.log(oracion); // "JavaScript es flexible"
```

Métodos que modifican arreglo original

No todos los métodos mantienen arreglo intacto. Algunos lo modifican.

Conviene conocer esta diferencia.

push

Agrega uno o más elementos al final.

```
const numeros = [1, 2, 3];
numeros.push(4);
console.log(numeros); // [1, 2, 3, 4]
```

pop

Elimina último elemento.

```
const numeros = [1, 2, 3];
const ultimo = numeros.pop();
console.log(ultimo); // 3
console.log(numeros); // [1, 2]
```

shift

Elimina primer elemento.

```
const nombres = ["Ana", "Luis", "Mora"];
const primero = nombres.shift();
console.log(primero); // "Ana"
console.log(nombres); // ["Luis", "Mora"]
```

unshift

Agrega uno o más elementos al inicio.

```
const nombres = ["Luis", "Mora"];
```

```
nombres.unshift("Ana");  
  
console.log(nombres); // ["Ana", "Luis", "Mora"]
```

Encadenamiento de métodos

Una gran ventaja de varios métodos de arreglos es que pueden encadenarse.

Ejemplo:

```
const numeros = [1, 2, 3, 4, 5, 6];  
  
const resultado = numeros  
  .filter(numero => numero % 2 === 0)  
  .map(numero => numero * 10);  
  
console.log(resultado); // [20, 40, 60]
```

En este caso:

1. filter deja solo números pares
2. map multiplica cada uno por 10

Comparación rápida

Método	¿Qué hace?	¿Qué devuelve?
forEach	recorre	undefined
map	transforma cada elemento	nuevo arreglo
filter	filtra según condición	nuevo arreglo
some	verifica si alguno cumple	boolean

Método	¿Qué hace?	¿Qué devuelve?
every	verifica si todos cumplen	boolean
find	busca primer elemento que cumple	elemento o undefined
findIndex	busca índice del primer elemento	índice o -1
reduce	acumula en un único resultado	valor final
includes	verifica existencia de valor	boolean
slice	copia una parte	nuevo arreglo
concat	une arreglos	nuevo arreglo
join	une en texto	string

¿Cuándo usar cada uno?

- usar **forEach** para recorrer y ejecutar acciones
- usar **map** para transformar datos
- usar **filter** para quedarse con algunos elementos
- usar **some** para saber si existe al menos uno
- usar **every** para validar conjunto completo
- usar **find** cuando se necesita primer elemento que coincida
- usar **reduce** cuando se quiere resumir todo en un valor

Cierre

Los métodos de arreglos permiten expresar operaciones muy comunes de forma más declarativa y legible que con muchos bucles manuales.

En este apunte vimos:

- cómo recorrer con `forEach`
- cómo transformar con `map`
- cómo filtrar con `filter`
- cómo validar con `some` y `every`
- cómo buscar con `find` y `findIndex`
- cómo acumular con `reduce`
- otros métodos frecuentes como `includes`, `slice`, `concat` y `join`

Dominar estos métodos ayuda a escribir código JavaScript más claro, reutilizable y fácil de mantener.

Introducción

Hasta ahora vimos JavaScript como lenguaje para trabajar con variables, condicionales, funciones y arreglos. Pero en desarrollo web una parte central del trabajo consiste en **interactuar con el documento HTML** que está cargado en el navegador.

Aquí aparecen dos ideas fundamentales:

- **DOM**, que representa la página como una estructura manipulable
- **eventos**, que permiten responder a acciones o cambios

Gracias a estas herramientas, JavaScript puede:

- leer contenido del documento
- modificar texto, atributos y estilos
- crear o eliminar elementos
- reaccionar ante clics, envíos de formularios o escritura en campos

En este apunte veremos cómo usar DOM y eventos, con ejemplos prácticos y con un foco especial en el trabajo con **formularios**, **listas** y **tablas**.

¿Qué es el DOM?

DOM significa **Document Object Model**.

Es una representación del documento HTML en forma de **árbol de objetos**. Cada elemento del documento pasa a ser un nodo que JavaScript puede consultar y modificar.

Por ejemplo, si tenemos este HTML:

```
<body>
```

```
<h1>Título</h1>
```

```
<p>Un párrafo</p>
```

```
</body>
```

el navegador construye una estructura donde:

- document representa documento completo
- body es un nodo dentro del documento
- h1 y p son nodos hijos de body

Eso permite hacer operaciones como:

- buscar elementos
- cambiar texto
- agregar nuevos nodos
- quitar elementos existentes

¿Por qué DOM es importante?

Porque HTML por sí solo define estructura inicial de página, pero el DOM permite que JavaScript la modifique durante ejecución.

Por ejemplo:

- actualizar un contador
- mostrar mensaje de error

- cargar datos en una lista
- agregar filas a una tabla

Objeto document

Punto de entrada principal al DOM suele ser document.

```
console.log(document);
```

Desde document se pueden buscar elementos y crear nuevos nodos.

Selección de elementos

Para manipular un elemento, primero hay que obtener una referencia a él.

getElementById

Busca elemento por su id.

```
<h1 id="titulo">Programación III</h1>
```

```
const titulo = document.getElementById("titulo");
```

```
console.log(titulo);
```

Es muy útil cuando elemento tiene identificador único.

querySelector

Busca **primer** elemento que coincida con selector CSS.

```
<p class="descripcion">Introducción al DOM</p>
```

```
const descripcion = document.querySelector(".descripcion");
```

```
console.log(descripcion);
```

También sirve con:

- etiquetas: "p"
- ids: "#titulo"

- clases: ".descripcion"

querySelectorAll

Busca **todos** los elementos que coinciden con selector CSS.

```
<li>HTML</li>
```

```
<li>CSS</li>
```

```
<li>JavaScript</li>
```

```
const items = document.querySelectorAll("li");
```

```
console.log(items);
```

Devuelve una colección que puede recorrerse:

```
const items = document.querySelectorAll("li");
```

```
items.forEach(item => {
```

```
  console.log(item.textContent);
```

```
});
```

Leer y modificar contenido

textContent

Permite leer o cambiar texto de un elemento.

```
<p id="mensaje">Texto original</p>
```

```
const mensaje = document.getElementById("mensaje");
```

```
console.log(mensaje.textContent); // Texto original
```

```
mensaje.textContent = "Texto actualizado";
```

innerHTML

Permite leer o escribir contenido HTML dentro de elemento.

```
<div id="contenedor"></div>
```

```
const contenedor = document.getElementById("contenedor");  
contenedor.innerHTML = "<strong>Contenido destacado</strong>";
```

Copiar

Diferencia entre textContent e innerHTML

- textContent trabaja con texto plano
- innerHTML interpreta etiquetas HTML

Si solo se quiere cambiar texto, en general textContent suele ser opción más simple y segura.

Trabajar con atributos

Leer atributo

```
<a id="enlace" href="https://example.com">Ir al sitio</a>
```

```
const enlace = document.getElementById("enlace");  
console.log(enlace.getAttribute("href"));
```

Modificar atributo

```
enlace.setAttribute("href", "https://github.com");
```

Acceso directo a propiedades frecuentes

Muchas propiedades se pueden leer y escribir directamente:

```
console.log(enlace.href);  
enlace.href = "https://www.wikipedia.org";
```

Clases y estilos

classList

Permite agregar, quitar o verificar clases CSS.

```
<p id="texto">Hola</p>
```

```
const texto = document.getElementById("texto");  
texto.classList.add("destacado");  
texto.classList.remove("destacado");  
texto.classList.toggle("activo");  
console.log(texto.classList.contains("activo"));
```

Estilos en línea con style

```
texto.style.color = "blue";  
texto.style.fontSize = "24px";
```

Aunque esto es posible, muchas veces conviene modificar clases y dejar estilos definidos en CSS.

Crear y agregar elementos

createElement

Permite crear un nuevo nodo.

```
const item = document.createElement("li");  
item.textContent = "Nuevo elemento";
```

appendChild

Agrega nodo al final de otro elemento.

```
<ul id="lista"></ul>
```

```
const lista = document.getElementById("lista");  
const item = document.createElement("li");  
item.textContent = "JavaScript";  
lista.appendChild(item);
```

append

También permite agregar contenido.

```
const lista = document.getElementById("lista");  
  
const item = document.createElement("li");  
  
item.textContent = "CSS";  
  
lista.append(item);
```

Eliminar elementos

```
<p id="aviso">Este mensaje será eliminado</p>
```

```
const aviso = document.getElementById("aviso");  
  
aviso.remove();
```

Eventos

Un **evento** es algo que ocurre en página y que JavaScript puede detectar.

Ejemplos:

- un clic
- una tecla presionada
- un formulario enviado
- un campo modificado
- una página terminada de cargar

Eventos en HTML

Existe forma clásica de declarar eventos directamente en HTML:

```
<button onclick="saludar()">Saludar</button>
```

```
function saludar() {
```

```
console.log("Hola");  
}
```

Esto funciona, pero **no suele ser forma recomendada** como estrategia principal.

¿Por qué no conviene abusar de eventos en HTML?

Porque:

- mezcla estructura con comportamiento
- dificulta mantenimiento
- hace más desordenado el HTML
- complica reutilización del código
- vuelve más difícil registrar varios manejadores
- limita organización en archivos JavaScript separados

Por eso, en código moderno se prefiere trabajar con:

- `addEventListener`
- `removeEventListener`
- funciones definidas en JavaScript

`addEventListener`

Es forma más habitual y recomendada para escuchar eventos.

Sintaxis general:

```
elemento.addEventListener("tipoDeEvento", manejador);
```

Ejemplo:

```
<button id="btnSaludar">Saludar</button>
```

```
const boton = document.getElementById("btnSaludar");  
boton.addEventListener("click", () => {  
  console.log("Hola");  
});
```

```
});
```

Ventajas de addEventListener

- separa HTML y JavaScript
- permite agregar varios listeners al mismo elemento
- mejora organización del código
- facilita mantenimiento
- puede removerse luego con removeEventListener

Eventos frecuentes

Algunos eventos muy usados son:

- click
- submit
- input
- change
- keydown
- DOMContentLoaded

click

Se dispara cuando persona usuaria hace clic.

```
<button id="btnContar">Sumar</button>
```

```
let contador = 0;
```

```
const boton = document.getElementById("btnContar");
```

```
boton.addEventListener("click", () => {
```

```
  contador++;
```

```
  console.log(contador);
```

```
});
```

input

Se dispara cada vez que cambia valor de un campo mientras se escribe.

```
<input id="nombre" type="text" />
```

```
const inputNombre = document.getElementById("nombre");  
inputNombre.addEventListener("input", event => {  
  console.log(event.target.value);  
});
```

change

Suele usarse cuando cambio queda confirmado, por ejemplo en select, checkbox, radio o cuando un input pierde foco.

```
<select id="turno">  
  <option>Mañana</option>  
  <option>Tarde</option>  
</select>
```

```
const turno = document.getElementById("turno");  
turno.addEventListener("change", event => {  
  console.log("Turno:", event.target.value);  
});
```

submit

Se dispara cuando se envía formulario.

```
<form id="formulario">  
  <input type="text" name="nombre" />  
  <button type="submit">Enviar</button>  
</form>
```

```
const formulario = document.getElementById("formulario");  
formulario.addEventListener("submit", event => {
```

```
event.preventDefault();  
console.log("Formulario enviado");  
});
```

Objeto event

Cuando ocurre evento, navegador entrega un objeto con información.

Ese objeto suele llamarse event o e.

```
const boton = document.getElementById("btnSaludar");  
boton.addEventListener("click", event => {  
  console.log(event);  
});
```

Propiedades y métodos frecuentes

event.target

Elemento que originó evento.

```
boton.addEventListener("click", event => {  
  console.log(event.target);  
});
```

event.preventDefault()

Evita comportamiento por defecto del navegador.

Muy común en formularios y enlaces.

```
formulario.addEventListener("submit", event => {  
  event.preventDefault();  
});
```

DOMContentLoaded

Este evento se dispara cuando HTML ya fue cargado y parseado.

Sirve para ejecutar código cuando nodos ya existen en DOM.

```
document.addEventListener("DOMContentLoaded", () => {  
  const titulo = document.getElementById("titulo");  
  console.log(titulo.textContent);  
});
```

Esto evita intentar acceder a elementos que todavía no fueron cargados.

Formularios y DOM

Los formularios son caso muy común de trabajo con eventos porque permiten:

- capturar datos
- validarlos
- agregarlos a estructura visual
- actualizar listas y tablas

Leer valores de un formulario

Ejemplo básico:

```
<form id="formPersona">  
  
<input id="nombre" type="text" placeholder="Nombre" />  
  
<input id="edad" type="number" placeholder="Edad" />  
  
<button type="submit">Guardar</button>  
  
</form>
```

```
const formPersona = document.getElementById("formPersona");  
const inputNombre = document.getElementById("nombre");  
const inputEdad = document.getElementById("edad");
```

```
formPersona.addEventListener("submit", event => {  
  event.preventDefault();  
  const nombre = inputNombre.value;  
  const edad = inputEdad.value;  
  console.log(nombre, edad);  
});
```

Limpiar formulario

Después de procesar datos, muchas veces conviene limpiar campos:

```
formPersona.addEventListener("submit", event => {  
  event.preventDefault();  
  console.log(inputNombre.value, inputEdad.value);  
  formPersona.reset();  
});
```

Ejemplo: cargar una lista desde un formulario

Supongamos que queremos ingresar nombres y agregarlos a una lista.

HTML

```
<form id="formLista">  
  <input id="inputItem" type="text" placeholder="Ingrese un elemento" />  
  <button type="submit">Agregar</button>  
</form>  
  
<ul id="listaElementos"></ul>
```

JavaScript

```
const formLista = document.getElementById("formLista");  
const inputItem = document.getElementById("inputItem");  
const listaElementos = document.getElementById("listaElementos");  
formLista.addEventListener("submit", event => {
```

```
event.preventDefault();

const valor = inputItem.value.trim();

if (valor === "") {
  return;
}

const li = document.createElement("li");

li.textContent = valor;

listaElementos.appendChild(li);

formLista.reset();

inputItem.focus();

});
```

¿Qué hace este ejemplo?

1. escucha envío del formulario
2. evita recarga de página con preventDefault()
3. toma valor del input
4. crea un li
5. agrega li a lista
6. limpia formulario

Ejemplo: cargar objetos en una lista

También podemos guardar datos en un arreglo y luego renderizar lista.

```
<form id="formTareas">

<input id="inputTarea" type="text" placeholder="Tarea" />

<button type="submit">Agregar tarea</button>

</form>

<ul id="listaTareas"></ul>
```

```
const tareas = [];  
  
const formTareas = document.getElementById("formTareas");  
  
const inputTarea = document.getElementById("inputTarea");  
  
const listaTareas = document.getElementById("listaTareas");  
  
function renderTareas() {  
  listaTareas.innerHTML = "";  
  
  tareas.forEach(tarea => {  
  
    const li = document.createElement("li");  
  
    li.textContent = tarea.nombre;  
  
    listaTareas.appendChild(li);  
  
  });  
}  
  
formTareas.addEventListener("submit", event => {  
  
  event.preventDefault();  
  
  const nombre = inputTarea.value.trim();  
  
  if (nombre === "") {  
  
    return;  
  
  }  
  
  tareas.push({ nombre: nombre });  
  
  renderTareas();  
  
  formTareas.reset();  
  
});
```

Acá aparece idea importante:

- una cosa es **datos**
- otra cosa es **cómo se muestran en DOM**

Ejemplo: cargar una tabla desde un formulario

Un caso muy habitual es tomar datos de formulario y agregarlos a una tabla.

HTML

```
<form id="formAlumno">

<input id="inputNombreAlumno" type="text" placeholder="Nombre" />

<input id="inputNotaAlumno" type="number" placeholder="Nota" />

<button type="submit">Agregar alumno</button>

</form>

<table border="1">

<thead>

<tr>

<th>Nombre</th>

<th>Nota</th>

</tr>

</thead>

<tbody id="tablaAlumnosBody"></tbody>

</table>
```

JavaScript

```
const alumnos = [];

const formAlumno = document.getElementById("formAlumno");

const inputNombreAlumno = document.getElementById("inputNombreAlumno");

const inputNotaAlumno = document.getElementById("inputNotaAlumno");

const tablaAlumnosBody = document.getElementById("tablaAlumnosBody");

function renderTabla() {

tablaAlumnosBody.innerHTML = "";

alumnos.forEach(alumno => {

const fila = document.createElement("tr");
```

```
const celdaNombre = document.createElement("td");
celdaNombre.textContent = alumno.nombre;
const celdaNota = document.createElement("td");
celdaNota.textContent = alumno.nota;
fila.appendChild(celdaNombre);
fila.appendChild(celdaNota);
tablaAlumnosBody.appendChild(fila);
});
}
formAlumno.addEventListener("submit", event => {
  event.preventDefault();
  const nombre = inputNombreAlumno.value.trim();
  const nota = inputNotaAlumno.value;
  if (nombre === "" || nota === "") {
    return;
  }
  alumnos.push({
    nombre: nombre,
    nota: Number(nota)
  });
  renderTabla();
  formAlumno.reset();
});
```

Alternativa: leer datos con FormData

Cuando formulario tiene varios campos, FormData puede resultar cómodo.

HTML

```
<form id="formProducto">

<input name="nombre" type="text" placeholder="Producto" />

<input name="precio" type="number" placeholder="Precio" />

<button type="submit">Guardar</button>

</form>
```

JavaScript

```
const formProducto = document.getElementById("formProducto");

formProducto.addEventListener("submit", event => {

event.preventDefault();

const datos = new FormData(formProducto);

const nombre = datos.get("nombre");

const precio = datos.get("precio");

console.log(nombre, precio);

});
```

Validaciones simples en formularios

Antes de agregar datos a lista o tabla, suele ser necesario validar.

```
formAlumno.addEventListener("submit", event => {

event.preventDefault();

const nombre = inputNombreAlumno.value.trim();

const nota = Number(inputNotaAlumno.value);

if (nombre === "") {

console.log("Nombre obligatorio");
```

```
return;

}

if (Number.isNaN(nota)) {
  console.log("Nota inválida");
  return;
}

});
```

removeEventListener

Si addEventListener agrega listener, removeEventListener permite quitarlo.

```
function saludar() {
  console.log("Hola");
}

const boton = document.getElementById("btnSaludar");

boton.addEventListener("click", saludar);

boton.removeEventListener("click", saludar);
```

Esto puede ser útil cuando comportamiento debe activarse o desactivarse según estado de aplicación.

Delegación de eventos

Cuando hay muchos elementos similares, en vez de agregar listener a cada uno se puede escuchar evento en contenedor común.

```
<ul id="lista">

<li>HTML</li>

<li>CSS</li>
```



```
</li>JavaScript</li>
```

```
</ul>
```

```
const lista = document.getElementById("lista");  
lista.addEventListener("click", event => {  
  if (event.target.tagName === "LI") {  
    console.log("Se hizo clic en:", event.target.textContent);  
  }  
});
```

Esto resulta útil cuando elementos se crean dinámicamente.

Buenas prácticas iniciales

- usar `addEventListener` en lugar de eventos escritos directamente en HTML
- separar estructura HTML, estilos CSS y lógica JavaScript
- usar `preventDefault()` en formularios cuando no se quiere recargar página
- validar datos antes de agregarlos al DOM
- preferir funciones de render cuando lista o tabla depende de un arreglo
- evitar repetir lógica de creación de nodos sin necesidad

Cierre

DOM permite que JavaScript lea y modifique documento HTML. Eventos permiten responder a acciones y cambios que ocurren en página.

En este apunte vimos:

- qué es DOM
- cómo seleccionar elementos

- cómo leer y modificar contenido
- cómo crear y eliminar nodos
- qué son eventos y cómo escucharlos
- por qué conviene preferir `addEventListener`
- cómo usar formularios para cargar listas y tablas

Estos conceptos son base para construir interfaces dinámicas e interactivas en la web.